Server-Client Setup for REDSQL

The following document outlines methods to install one or more red servers and clients. The end result is an environment sufficient for running redsql.

1. First create your cluster and ensure its running by whatever method you prefer. I would suggest the method outlined in

   ```
   tools/containers/README.red-dev-containers.md
   ```

   The method outlined in README.red-dev-containers.md installs the redsetup package and performs a realm creation. It then switches the container binaries to run directly from your build tree (ie. sandbox). This is beneficial if you are constantly changing code and rebuilding. By directly running your server binaries from your build tree, you do not have to constantly redeploy your server. Instead just stop and restart it and you'll be running with your new binaries.

   For developers, it is also possible to utilize the tools/scripts/red-deploy script to create a cluster on red-devhead{1|2}.

2. redsql is intended to run in a red client and requires a cluster, tenant, subtenant, and dataset. We will be creating these after you get your packages installed and your realm created (see README.red-dev-containers.md)

3. Other packages:

   It is possible to run the red client directly on a server node or as a separate client node. If you intend to run a client directly on a server then there is no need to install the redcli package because the redcli binaries are ALSO in the redsetup package. In fact the install of redcli package will fail if the redsetup is present. The redtools package CAN (you don't necessarily need it) be installed on either client or server nodes.

   If you need the redcli package (running disjoint from server) then you can retrieve it in the same manner as how you retrieved redsetup in README.red-dev-containers.md. The package is just named slightly different. Here is a direct transcription of how to get the redcli package using the same method as redsetup.

```
export BASE_PKG_URL="https://storage.googleapis.com/ddn-redsetup-public" && export RELEASE_TYPE="-engr" && \
export TARGET_ARCH="$(dpkg --print-architecture)" && \
export REL_DIST_PATH="ubuntu/24.04" && \
export REL_PKG_URL="${BASE_PKG_URL}/releases${RELEASE_TYPE}/${REL_DIST_PATH}" && \
export RED_VER=$(wget -q  -O - $REL_PKG_URL/RELEASE_VERSION_${TARGET_ARCH}.txt?cache-time="$(date +$s)")
wget $REL_PKG_URL/redcli_"${RED_VER}"_"${TARGET_ARCH}${RELEASE_TYPE}".deb?cache-time="$(date +$s)" -O
/tmp/redcli.deb && \
sudo apt install -y /tmp/redcli.deb
```

4. Now ensure you have performed redsetup on your realm-entry and non-realm-entry server nodes as well as creating a realm configuration. This is step (4), (5), and (6) in README.red-dev-containers.md. You're servers are mostly configured at this point but we'll still need a cluster as well as tenants.

5. Cluster and Tenants

I believe you can create a cluster and tenants from either a server or client because it is merely an invocation of redcli that talks to the server. In the following example we're creating the cluster, tenant, subtenant on a server node (e.g. realm-entry node).

```
export REALM_ENTRY=myname-agent-1
export CLUSTER=mycluster
export TENANT=ten1
export SUBTENANT=sub1
export PASSWORD=abcdabcd
export DATASET=fs1

redcli user login realm_admin --password ${PASSWORD}
redcli cluster create ${CLUSTER}
redcli cluster show --status && redcli cluster show --health

# realm admin can create tenants
redcli tenant create ${TENANT}

# grant myself access to tenants I created
redcli user grant realm_admin ${TENANT}

# create the subtenant
redcli subtenant create ${SUBTENANT} -t ${TENANT}

# grant myself access to the subtenant
redcli user grant realm_admin ${TENANT}/${SUBTENANT}

# Now create a dataset and associate it with your tenant
redcli dataset create ${DATASET} --tenant ${TENANT} --subtenant ${SUBTENANT}
```

6. Setting up a client

Ensure you are either setting up a client on a server node with redsetup package installed OR you are on a separate node and have installed the redcli package. You will be connecting (via redcli) to a server in your cluster to do this (such as the realm-entry server).

Note if you are ALREADY on a server (setting up a client on a server) then it may be possible to not use the --server syntax. However on Oct 2, 2024 there seems to be a bug when not explicitly identifying the server in the following commands.

```
export REDCLI_SERVER=https://${REALM_ENTRY}/redapi/v1
redcli user login realm_admin --password ${PASSWORD} --server ${REDCLI_SERVER}
redcli client setup                                  --server ${REDCLI_SERVER}

```

The prior commands will log you in and (unless explicitly specified via command options) will automatically select your cluster and tenants.

Example output:

```
9:42PM INF Auto Selecting cluster: mycluster
9:42PM INF Auto Selecting tenant: ten1
9:42PM INF Auto Selecting subtenant: sub1
9:42PM INF Certificates and keys saved at /etc/red/certs. Be careful!
9:42PM INF RED client setup complete, Hint: run 'source /nfs_home/dcraft/.config/red/redrc' to set RED
environment.
```

At this point you should source the environment file specfied in the prior line

```
source $HOME/.config/red/redrc
```

7. redsql works against a RED dataset. You can utilize redsql as a comand line tool and query regular sql statements or explore the file system metadata of a redfs. For both cases, first you have to make sure that you have your client environment set up. Check the paragraph above on how to do this.

   Note, the following commands assume that you have set the tenant, subtenant, and dataset environment variables to the desired values.

   After the client set up is completed you can start the redsql with the following command.

   ```
   # Launch redsql
   redsql --dataset ${DATASET}
   ```

   The file system metadata can be explored with a special table, redfs. To use the redfs table, you should create a file system and populate it with some data to get interesting SQL results.

   The following command will create a redfs file system. Note that the dataset must already exist.

   ```
   # Format the file system.
   mkfs.redfs -T ${TENANT} -S ${SUBTENANT} -D ${DATASET} 2>/dev/null
   ```

   Then, you will have to create the mount point directory

   ```
   # create a mount point
   cd ~
   mkdir mnt
   ```

   To mount the file system you can use the following command:

   ```
   # Mount the file system
   redfs -T ${TENANT} -S ${SUBTENANT} -D ${DATASET}  mnt || echo "failed to mount redfs"
   
   ```

   After that you can populate the file system with some data so that the redsql can query it.

8. Installing postgres

   We utilize the postgres database front end for managing certain SELECT operations. As such we provide an extension module (foreign data wrapper module) to allow postgres to talk to RED.

   We have currently tested our foreign data wrapper module with postgresql-16. If you have deployed your development environment and RED server/client instances with Ubuntu-24-04, then the postgresql 16 packages and may be installed as:

   ```
   sudo apt install postgresql postgresql-server-dev-16
   ```

   If you have an earlier version of Ubuntu deployed then you may need to point to the PostgreSQL repository and

update the repository signing key.

```
# Add the PostgreSQL 16 repository:
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'

# Import the repository signing key:
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/postgresql.gpg

# Update the package list:
sudo apt update

# Install postgresql 16 and server development
sudo apt install postgresql-16 postgresql-server-dev-16
```